Essential Functional Analysis

Prerequisites for the formalisation of Vern Paulsen's lectures on quantum computing

Dustin Bryant

October 27, 2026

Abstract

General facts about complex matrices that the formalised lectures (session `Quantum_Computing_by_VP`) build on: sums, adjoints, unitary matrices and Kronecker products, linear-space isomorphisms, and matrices of fixed dimension as a type, with the Hilbert–Schmidt norm (a normed space for the Hahn–Banach theorem of `HOL-Hahn_Banach`) and the structure of a Euclidean space (so that `HOL-Analysis` applies). Nothing here depends on the lectures.

Contents

1 Complex numbers and scalar multiples **2**

2 The adjoint **2**

3 Sums of vectors **2**

4 Products and sums of matrices **3**
4.1 Algebra of *foldr*-sums of matrices 5

5 Finite sums and index arithmetic **6**

6 Linear maps on a set of matrices **7**

7 Linear-space isomorphisms **8**

8 Unitary algebra **9**

9 Kronecker products **10**
9.1 Tensor of Identities 10
9.2 Tensor Adjoint 10
9.3 Mixed-Product Property 10
9.4 Corollaries of the Mixed-Product Property 10
9.5 Tensor Unitarity 10
9.6 Tensor products of matrices 11

10 Typed-dimension matrix wrapper **12**

11 Typeclass instances **12**

12 Real scalar multiplication and Hahn-Banach overloading **14**

13 Real-vectorspace locale on the full type **14**

14 The Hilbert-Schmidt norm on (*complex*, *'m*, *'n*) *lmat* **14**

15 Subspace, seminorm, normed-vectorspace instances **15**

16 Complex scalar multiplication on lmat **15**

17 Overview **17**

18 Real scalar multiplication and *real-vector* instance **17**

19 Entries of sums and differences **17**

20 Distance, uniformity, open: *metric-space* instance **18**

21 Norm and *real-normed-vector* instance **18**

22 Real inner product **19**

23 Basis: matrix units carrying a basis vector **19**

24 Basis set **20**

25 Euclidean space instance **21**

26 Heine-Borel **21**

27 Dimension **21**

28 Component-wise bounds via the L2 norm **21**

29 Convergence: lmat sequence convergence implies entry convergence **22**

30 Entries as continuous functions; wrapping **22**

The gate names of Isabelle_Marries_Dirac would capture the letters X, Y, Z, H, S, T used as variables in the lectures.

hide-const (**open**) *Quantum.X Quantum.Y Quantum.Z Quantum.H Quantum.S Quantum.T*

1 Complex numbers and scalar multiples

lemma *cnj-mult-self-cmod*: *cnj z * z = complex-of-real ((cmod z)²)*
⟨*proof*⟩

lemma *Re-cnj-mult-self*: *Re (cnj z * z) = (cmod z)²*
⟨*proof*⟩

lemma *smult-smult-mat*:
fixes *A* :: *complex Matrix.mat*
shows *a ·m (b ·m A) = (a * b) ·m A*
⟨*proof*⟩

lemma *zero-smult-mat-carrier*:
fixes *A* :: *complex Matrix.mat*
assumes *A ∈ carrier-mat m k*
shows *0 ·m A = 0m m k*
⟨*proof*⟩

2 The adjoint

The adjoint of an $m \times n$ matrix is $n \times m$.

lemma *dagger-carrier*:
assumes *M ∈ carrier-mat m n*
shows *(M†) ∈ carrier-mat n m*
⟨*proof*⟩

lemma *dagger-mult-diag-inner*:
fixes *Y* :: *complex Matrix.mat*
assumes *Yc*: *Y ∈ carrier-mat p q* **and** *j*: *j < q*
shows *((Y†) * Y) $$ (j,j) = ⟨Matrix.col Y j | Matrix.col Y j⟩*
⟨*proof*⟩

3 Sums of vectors

Finite sums of vectors, entrywise.

definition *vec-sum* :: *nat ⇒ 'c set ⇒ ('c ⇒ complex Matrix.vec) ⇒ complex Matrix.vec* **where**
vec-sum n Js w = Matrix.vec n (λi. sum (λJ. w J $ i) Js)

lemma *vec-sum-dim* [*simp*]: *dim-vec (vec-sum n Js w) = n*
⟨*proof*⟩

lemma *vec-sum-index* [*simp*]: *i < n ⟹ vec-sum n Js w $ i = sum (λJ. w J $ i) Js*
⟨*proof*⟩

lemma *vec-sum-cong*:
assumes *⋀J. J ∈ Js ⟹ w J = w' J*
shows *vec-sum n Js w = vec-sum n Js w'*
⟨*proof*⟩

lemma *mult-mat-vec-vec-sum*:
assumes *Ac*: $A \in$ *carrier-mat m n* **and** *fin*: *finite Js*
and *wd*: $\bigwedge J.\ J \in Js \Longrightarrow$ *dim-vec* $(w\ J) = n$
shows $A *_v$ *vec-sum n Js* $w =$ *vec-sum m Js* $(\lambda J.\ A *_v w\ J)$
⟨*proof*⟩

lemma *mult-mat-vec-smult-vec*:
fixes $A ::$ *complex Matrix.mat* **and** $v ::$ *complex Matrix.vec*
assumes *Ac*: $A \in$ *carrier-mat m n* **and** *vd*: *dim-vec* $v = n$
shows $A *_v (c \cdot_v v) = c \cdot_v (A *_v v)$
⟨*proof*⟩

4 Products and sums of matrices

lemma *mat-mult-entry*:
fixes $A\ B ::$ *complex Matrix.mat*
assumes $A \in$ *carrier-mat r s* $B \in$ *carrier-mat s t* $k < r\ j < t$
shows $(A * B)$ \$\$ $(k,j) = (\sum l{<}s.\ A$ \$\$ $(k,l) * B$ \$\$ $(l,j))$
⟨*proof*⟩

The same with the sum over $\{0..<q\}$; square matrices that agree on every vector are equal; a non-zero matrix has a non-zero entry.

lemma *mat-mult-index-sum*:
assumes *A-c*: $A \in$ *carrier-mat p q*
and *B-c*: $B \in$ *carrier-mat q s*
and *r-lt*: $r < p$ **and** *c-lt*: $c < s$
shows $(A * B)$ \$\$ $(r,\ c) = (\sum k = 0..{<}q.\ A$ \$\$ $(r,\ k) * B$ \$\$ $(k,\ c))$
⟨*proof*⟩

lemma *mat-eq-of-mult-vec-eq*:
fixes $M\ N ::$ *complex Matrix.mat*
assumes *M-c*: $M \in$ *carrier-mat k k*
and *N-c*: $N \in$ *carrier-mat k k*
and *eq*: $\bigwedge x.\ x \in$ *carrier-vec* $k \Longrightarrow M *_v x = N *_v x$
shows $M = N$
⟨*proof*⟩

lemma *nonzero-entry*:
fixes $A ::$ *complex Matrix.mat*
assumes *A*: $A \in$ *carrier-mat m k* **and** *nz*: $A \neq 0_m\ m\ k$
obtains $r\ s$ **where** $r < m\ s < k\ A$ \$\$ $(r,s) \neq 0$
⟨*proof*⟩

An independent frame cancels: if M^*M is invertible, $MAM^* = 0$ forces $A = 0$.

lemma *frame-cancel*:
fixes $M\ A\ B ::$ *complex Matrix.mat*
assumes *M*: $M \in$ *carrier-mat m k* **and** *A*: $A \in$ *carrier-mat k k* **and** *B*: $B \in$ *carrier-mat k k*
and *BG*: $B * (M^\dagger * M) = 1_m\ k$ **and** *GB*: $(M^\dagger * M) * B = 1_m\ k$
and *z*: $M * A * (M^\dagger) = 0_m\ m\ m$
shows $A = 0_m\ k\ k$
⟨*proof*⟩

lemma *foldr-add-mat-carrier*:
fixes $f :: 'a \Rightarrow$ *complex Matrix.mat*
assumes $\forall x \in$ *set xs.* $f\ x \in$ *carrier-mat n n*
shows *foldr* $(\lambda x\ acc.\ f\ x +\ acc)\ xs\ (0_m\ n\ n) \in$ *carrier-mat n n*
⟨*proof*⟩

lemma *foldr-add-mat-index*:
fixes $f :: 'a \Rightarrow$ *complex Matrix.mat*
assumes $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat n n* **and** $i < n$ **and** $j < n$
shows *foldr* $(\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ n\ n)$ \$\$ $(i, j) =$ *sum-list* $(map\ (\lambda x.\ f\ x$ \$\$ $(i, j))\ xs)$
⟨*proof*⟩

lemma *foldr-add-mat-carrier-rect*:
assumes $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat p q*
shows *foldr* $(\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ p\ q) \in$ *carrier-mat p q*
⟨*proof*⟩

lemma *foldr-add-mat-index-rect*:
assumes $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat p q* **and** $i < p$ **and** $j < q$
shows *foldr* $(\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ p\ q)$ \$\$ $(i,j) = (\sum x \leftarrow xs.\ f\ x$ \$\$ $(i,j))$
⟨*proof*⟩

Carrier-preservation in a different dimension pair.

lemma *foldr-add-carrier-dn*:
fixes $f :: 'a \Rightarrow$ *complex Matrix.mat*
assumes $\forall x \in$ *set xs*. $f\ x \in$ *carrier-mat d n*
shows *foldr* $(\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ d\ n) \in$ *carrier-mat d n*
⟨*proof*⟩

lemma *foldr-add-mat-cong*:
fixes $f\ g :: 'a \Rightarrow$ *complex Matrix.mat*
assumes $\bigwedge x.\ x \in$ *set xs* $\Longrightarrow f\ x = g\ x$
shows *foldr* $(\lambda x\ acc.\ f\ x + acc)\ xs\ z =$ *foldr* $(\lambda x\ acc.\ g\ x + acc)\ xs\ z$
⟨*proof*⟩

Sums over $[0..<N]$, and their entries.

lemma *foldr-upt-add-carrier*:
fixes $f ::$ *nat* $\Rightarrow$ *complex Matrix.mat*
assumes $\forall k < N.\ f\ k \in$ *carrier-mat d d*
shows *foldr* $(\lambda k\ acc.\ f\ k + acc)\ [0..<N]\ (0_m\ d\ d) \in$ *carrier-mat d d*
⟨*proof*⟩

lemma *foldr-upt-add-acc-index*:
fixes $f ::$ *nat* $\Rightarrow$ *complex Matrix.mat*
assumes *carr*: $\forall k < N.\ f\ k \in$ *carrier-mat d d*
and *z-c*: $z \in$ *carrier-mat d d*
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows *foldr* $(\lambda k\ acc.\ f\ k + acc)\ [0..<N]\ z$ \$\$ $(r,\ c)$
$= (\sum k = 0..<N.\ f\ k$ \$\$ $(r,\ c)) + z$ \$\$ $(r,\ c)$
⟨*proof*⟩

lemma *foldr-upt-add-index*:
fixes $f ::$ *nat* $\Rightarrow$ *complex Matrix.mat*
assumes *carr*: $\forall k < N.\ f\ k \in$ *carrier-mat d d*
and *r-lt*: $r < d$ **and** *c-lt*: $c < d$
shows *foldr* $(\lambda k\ acc.\ f\ k + acc)\ [0..<N]\ (0_m\ d\ d)$ \$\$ $(r,\ c)$
$= (\sum k = 0..<N.\ f\ k$ \$\$ $(r,\ c))$
⟨*proof*⟩

lemma *foldr-upt-add-acc-index-dn*:
fixes $f ::$ *nat* $\Rightarrow$ *complex Matrix.mat*
assumes *carr*: $\forall k < N.\ f\ k \in$ *carrier-mat d n*
and *z-c*: $z \in$ *carrier-mat d n*

and *r-lt*: $r < d$ **and** *c-lt*: $c < n$
shows *foldr* $(\lambda k\ acc.\ f\ k + acc)\ [0..<N]\ z\ \$\$\ (r,\ c)$
$= (\sum k = 0..<N.\ f\ k\ \$\$\ (r,\ c)) + z\ \$\$\ (r,\ c)$
⟨*proof*⟩

lemma *foldr-smult-nth-index*:
fixes $c :: nat \Rightarrow real$ **and** $Bs :: complex\ Matrix.mat\ list$
assumes *Bs-c*: $\forall B \in set\ Bs.\ B \in carrier\text{-}mat\ n\ n$
and *r-lt*: $r < n$ **and** *c-lt*: $c1 < n$
shows *foldr* $(\lambda k\ acc.\ complex\text{-}of\text{-}real\ (c\ k) \cdot_m (Bs\ !\ k) + acc)$
$[0..<length\ Bs]\ (0_m\ n\ n)\ \$\$\ (r,\ c1)$
$= (\sum k = 0..<length\ Bs.$
$complex\text{-}of\text{-}real\ (c\ k) * (Bs\ !\ k)\ \$\$\ (r,\ c1))$
⟨*proof*⟩

lemma *foldr-smult-index*:
assumes *Bs*: $\forall B \in set\ Bs.\ B \in carrier\text{-}mat\ n\ m$ **and** r: $r < n$ **and** s: $s < m$
shows *foldr* $(\lambda k\ acc.\ c\ k \cdot_m (Bs\ !\ k) + acc)\ [0..<length\ Bs]\ (0_m\ n\ m)\ \$\$\ (r,s)$
$= (\sum k<length\ Bs.\ c\ k * Bs\ !\ k\ \$\$\ (r,s))$
⟨*proof*⟩

4.1 Algebra of *foldr*-sums of matrices

lemma *foldr-add-mat-mult-left*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes *Ac*: $A \in carrier\text{-}mat\ m\ n$ **and** *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ n\ p$
shows $A * foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ n\ p) = foldr\ (\lambda x\ acc.\ A * f\ x + acc)\ xs\ (0_m\ m\ p)$
⟨*proof*⟩

lemma *foldr-add-mat-mult-right*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes *Bc*: $B \in carrier\text{-}mat\ n\ p$ **and** *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ m\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ m\ n) * B = foldr\ (\lambda x\ acc.\ f\ x * B + acc)\ xs\ (0_m\ m\ p)$
⟨*proof*⟩

lemma *foldr-add-mat-dagger*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$
assumes $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ m\ n$
shows $(foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ m\ n))^\dagger = foldr\ (\lambda x\ acc.\ (f\ x)^\dagger + acc)\ xs\ (0_m\ n\ m)$
⟨*proof*⟩

lemma *foldr-add-mat-zero*:
$foldr\ (\lambda x\ acc.\ (0_m\ m\ n :: complex\ Matrix.mat) + acc)\ xs\ (0_m\ m\ n) = 0_m\ m\ n$
⟨*proof*⟩

lemma *foldr-add-mat-delta*:
fixes $f :: nat \Rightarrow complex\ Matrix.mat$
assumes *fc*: $\forall x \in set\ xs.\ f\ x \in carrier\text{-}mat\ m\ n$ **and** *d*: *distinct xs* **and** *i*: $i \in set\ xs$
shows $foldr\ (\lambda x\ acc.\ (if\ x = i\ then\ f\ x\ else\ 0_m\ m\ n) + acc)\ xs\ (0_m\ m\ n) = f\ i$
⟨*proof*⟩

lemma *foldr-add-mat-smult-pull*:
fixes $f :: 'a \Rightarrow complex\ Matrix.mat$ **and** $c :: 'a \Rightarrow complex$
assumes *fc*: $\forall x \in set\ xs.\ f\ x = c\ x \cdot_m M$ **and** *Mc*: $M \in carrier\text{-}mat\ m\ n$
shows $foldr\ (\lambda x\ acc.\ f\ x + acc)\ xs\ (0_m\ m\ n) = sum\text{-}list\ (map\ c\ xs) \cdot_m M$
⟨*proof*⟩

lemma *foldr-list-as-index*:

fixes g :: *complex Matrix.mat* ⇒ *complex Matrix.mat*
shows *foldr* (λ*A acc. g A* + *acc*) *As z* = *foldr* (λ*l acc. g* (*As* ! *l*) + *acc*) [*0*..<*length As*] *z*
⟨*proof*⟩

definition *msum* :: *nat* ⇒ *nat* ⇒ (′*a* ⇒ *complex Matrix.mat*) ⇒ ′*a list* ⇒ *complex Matrix.mat* **where**
msum r c g xs = *foldr* (λ*x acc. g x* + *acc*) *xs* (*0*$_m$ *r c*)

lemma *msum-Nil* [*simp*]: *msum r c g* [] = *0*$_m$ *r c*
⟨*proof*⟩

lemma *msum-Cons* [*simp*]: *msum r c g* (*x* # *xs*) = *g x* + *msum r c g xs*
⟨*proof*⟩

lemma *msum-carrier*: ∀ *x* ∈ *set xs. g x* ∈ *carrier-mat r c* ⟹ *msum r c g xs* ∈ *carrier-mat r c*
⟨*proof*⟩

lemma *msum-index*:
assumes ∀ *x* ∈ *set xs. g x* ∈ *carrier-mat r c* **and** *i* < *r* **and** *j* < *c*
shows *msum r c g xs* $$ (*i*,*j*) = (∑ *x*←*xs. g x* $$ (*i*,*j*))
⟨*proof*⟩

Every *msum* has its stated dimensions; multiplying a sum on the right.

lemma *msum-dims* [*simp*]: *dim-row* (*msum r c g xs*) = *r dim-col* (*msum r c g xs*) = *c*
⟨*proof*⟩

lemma *msum-mult-right*:
assumes ∀ *x* ∈ *set xs. g x* ∈ *carrier-mat r c* **and** *B* ∈ *carrier-mat c p*
shows *msum r c g xs* ∗ *B* = *msum r p* (λ*x. g x* ∗ *B*) *xs*
⟨*proof*⟩

5 Finite sums and index arithmetic

lemma *sum-list-product-upt*:
(∑ *x*←*List.product* [*0*..<*a*] [*0*..<*b*]. *h x*) = (∑ *i*<*a*. ∑ *j*<*b*. *h* (*i*,*j*))
⟨*proof*⟩

lemma *sum-delta2*:
fixes *a d b e* :: *nat*
assumes *a*: *a* < *d* **and** *b*: *b* < *e*
shows (∑ *i*<*d*. ∑ *j*<*e*. *if i* = *a* ∧ *j* = *b then v else 0*) = *v*
⟨*proof*⟩

Concatenating rows of equal length, and the reindexing $\kappa = ik + j$.

lemma *concat-uniform-length*:
assumes ∀ *xs* ∈ *set L. length xs* = *m*
shows *length* (*concat L*) = *length L* ∗ *m*
⟨*proof*⟩

lemma *concat-uniform-nth*:
assumes *uniform*: ∀ *xs* ∈ *set L. length xs* = *m*
and *i-lt*: *i* < *length L*
and *j-lt*: *j* < *m*
shows *concat L* ! (*i* ∗ *m* + *j*) = *L* ! *i* ! *j*
⟨*proof*⟩

lemma *sum-split-prod-atLeastLessThan*:
fixes *f* :: *nat* ⇒ ′*a*::*comm-monoid-add*

shows $(\sum \kappa = 0..<N * k.\ f\ \kappa) = (\sum i = 0..<N.\ \sum j = 0..<k.\ f\ (i * k + j))$
⟨*proof*⟩

Sums over pairs of indices (i, j) with $i, j < k$.

lemma *msum-pairs-carrier*:
assumes $\bigwedge i\ j.\ i < k \Longrightarrow j < k \Longrightarrow g\ i\ j \in$ *carrier-mat r c*
shows *msum r c* $(\lambda(i,\ j).\ g\ i\ j)$ (*List.product* $[0..<k]$ $[0..<k]$) ∈ *carrier-mat r c*
⟨*proof*⟩

lemma *msum-pairs-index*:
assumes *g*: $\bigwedge i\ j.\ i < k \Longrightarrow j < k \Longrightarrow g\ i\ j \in$ *carrier-mat r c* **and** *a*: $a < r$ **and** *b*: $b < c$
shows *msum r c* $(\lambda(i,\ j).\ g\ i\ j)$ (*List.product* $[0..<k]$ $[0..<k]$) $$ $(a,\ b)$
$= (\sum i{<}k.\ \sum j{<}k.\ g\ i\ j\ \$\$\ (a,\ b))$
⟨*proof*⟩

lemma *msum-pairs-mult-right*:
assumes *g*: $\bigwedge i\ j.\ i < k \Longrightarrow j < k \Longrightarrow g\ i\ j \in$ *carrier-mat r c* **and** *B*: $B \in$ *carrier-mat c p*
shows *msum r c* $(\lambda(i,\ j).\ g\ i\ j)$ (*List.product* $[0..<k]$ $[0..<k]$) $* B$
$=$ *msum r p* $(\lambda(i,\ j).\ g\ i\ j * B)$ (*List.product* $[0..<k]$ $[0..<k]$)
⟨*proof*⟩

lemma *msum-upt-index*:
assumes $\bigwedge x.\ x < m \Longrightarrow g\ x \in$ *carrier-mat r c* **and** $a < r$ **and** $b < c$
shows *msum r c g* $[0..<m]$ $$ $(a,\ b) = (\sum x{<}m.\ g\ x\ \$\$\ (a,\ b))$
⟨*proof*⟩

lemma *idx8*: $k < (8{::}nat) \Longrightarrow k = 0 \vee k = 1 \vee k = 2 \vee k = 3 \vee k = 4 \vee k = 5 \vee k = 6 \vee k = 7$
⟨*proof*⟩

6 Linear maps on a set of matrices

Linearity of a map defined on a set of $n \times n$ matrices (an operator system, from Day 24 on) with values in M_d.

definition *lin-on* :: *nat* ⇒ *nat* ⇒ *complex Matrix.mat set* ⇒ (*complex Matrix.mat* ⇒ *complex Matrix.mat*) ⇒ *bool* **where**
lin-on n d OS $\Phi \longleftrightarrow (\forall A \in OS.\ \Phi\ A \in$ *carrier-mat d d*$) \wedge (\forall A \in OS.\ \forall B \in OS.\ \Phi\ (A + B) = \Phi\ A + \Phi\ B)$
$\wedge\ (\forall A \in OS.\ \forall c.\ \Phi\ (c \cdot_m A) = c \cdot_m \Phi\ A)$

On a set closed under sums and containing 0, such a map commutes with finite sums.

lemma *msum-closed*:
assumes *S0*: $0_m\ n\ n \in S$ **and** *S*: $\bigwedge A\ B.\ A \in S \Longrightarrow B \in S \Longrightarrow A + B \in S$
and *g*: $\forall x \in$ *set xs*. $g\ x \in S$
shows *msum n n g xs* ∈ *S*
⟨*proof*⟩

lemma *lin-on-msum-closed*:
assumes *L*: *lin-on n d S* Φ **and** *S0*: $0_m\ n\ n \in S$ **and** *S*: $\bigwedge A\ B.\ A \in S \Longrightarrow B \in S \Longrightarrow A + B \in S$
and *g*: $\forall x \in$ *set xs*. $g\ x \in S$
shows Φ (*msum n n g xs*) = *msum d d* $(\lambda x.\ \Phi\ (g\ x))$ *xs*
⟨*proof*⟩

lemma *lin-on-smult*:
assumes *lin-on n d S F*
shows *lin-on n d S* $(\lambda X.\ a \cdot_m F\ X)$
⟨*proof*⟩

7 Linear-space isomorphisms

Matrix spaces are represented by carrier predicates on the single type *complex mat*. The record below packages a carrier with its zero, addition and scalar multiplication, so that statements such as $L(\mathcal{H}_A) \otimes L(\mathcal{H}_B) \cong L(\mathcal{H}_A \otimes \mathcal{H}_B)$ (Day 15) can be made as explicit linear isomorphisms with explicit inverses.

record ('*a*, '*v*) *linear-structure* =
  ls-carrier :: '*v set*
  ls-zero :: '*v*
  ls-add :: '*v* ⇒ '*v* ⇒ '*v*
  ls-smul :: '*a* ⇒ '*v* ⇒ '*v*

definition *linear-structure-map* ::
  ('*a*, '*v*) *linear-structure* ⇒ ('*a*, '*w*) *linear-structure* ⇒ ('*v* ⇒ '*w*) ⇒ *bool* **where**
  linear-structure-map V W f ≡
    f (*ls-zero V*) = *ls-zero W* ∧
    (∀ *x* ∈ *ls-carrier V*. *f x* ∈ *ls-carrier W*) ∧
    (∀ *x* ∈ *ls-carrier V*. ∀ *y* ∈ *ls-carrier V*.
      f (*ls-add V x y*) = *ls-add W* (*f x*) (*f y*)) ∧
    (∀ *c x*. *x* ∈ *ls-carrier V* ⟶
      f (*ls-smul V c x*) = *ls-smul W c* (*f x*))

definition *is-isomorphism-between* ::
  ('*a*, '*v*) *linear-structure* ⇒ ('*a*, '*w*) *linear-structure* ⇒ ('*v* ⇒ '*w*) ⇒ *bool* **where**
  is-isomorphism-between V W f ≡
    linear-structure-map V W f ∧
    (∃ *g*. *linear-structure-map W V g* ∧
        (∀ *x* ∈ *ls-carrier V*. *g* (*f x*) = *x*) ∧
        (∀ *y* ∈ *ls-carrier W*. *f* (*g y*) = *y*))

definition *is-isomorphic-to* ::
  ('*a*, '*v*) *linear-structure* ⇒ ('*a*, '*w*) *linear-structure* ⇒ *bool* **where**
  is-isomorphic-to V W ≡ ∃ *f*. *is-isomorphism-between V W f*

lemma *is-isomorphism-betweenD-map*:
  assumes *is-isomorphism-between V W f*
  shows *linear-structure-map V W f*
  ⟨*proof*⟩

lemma *is-isomorphism-betweenD-inverse*:
  assumes *is-isomorphism-between V W f*
  obtains *g* **where** *linear-structure-map W V g*
    and ⋀*x*. *x* ∈ *ls-carrier V* ⟹ *g* (*f x*) = *x*
    and ⋀*y*. *y* ∈ *ls-carrier W* ⟹ *f* (*g y*) = *y*
  ⟨*proof*⟩

The example used throughout: $M_{m,n}$ with entrywise addition and scaling.

definition *mat-linear-structure* :: *nat* ⇒ *nat* ⇒ (*complex*, *complex Matrix.mat*) *linear-structure* **where**
  mat-linear-structure m n ≡
    (| *ls-carrier* = *carrier-mat m n*,
      ls-zero = 0_m *m n*,
      ls-add = (λ*x y*. *x* + *y*),
      ls-smul = (λ*c x*. *c* ·$_m$ *x*) |)

hide-const (**open**) *Quantum.X Quantum.Y Quantum.Z Quantum.H Quantum.S Quantum.T*
declare *cauchy-schwarz-eq* [*simp del*] *cauchy-schwarz-col* [*simp del*] *eq-from-inner-prod* [*simp del*]

8 Unitary algebra

Library-level consequences of unitarity ($U^\dagger U = I$ and $UU^\dagger = I$ in the library's definition) used throughout; Paulsen's Day 3 characterisations of unitaries are in theory Day03_Unitary_Matrices.

lemma *unitary-square-dim*:
  assumes *uA*: *unitary A*
  shows *dim-row A* = *dim-col A*
⟨*proof*⟩

lemma *unitary-inv-l-dimcol*:
  assumes *uA*: *unitary A*
  shows $(A^\dagger) * A = 1_m$ (*dim-col A*)
  ⟨*proof*⟩

lemma *unitary-inv-r-dimrow*:
  assumes *uA*: *unitary A*
  shows $A * (A^\dagger) = 1_m$ (*dim-row A*)
  ⟨*proof*⟩

lemma *unitary-inv-l-dimrow*:
  assumes *uA*: *unitary A*
  shows $(A^\dagger) * A = 1_m$ (*dim-row A*)
  ⟨*proof*⟩

lemma *unitary-inv-r-dimcol*:
  assumes *uA*: *unitary A*
  shows $A * (A^\dagger) = 1_m$ (*dim-col A*)
  ⟨*proof*⟩

lemma *unitary-carrier-square-col*:
  assumes *uA*: *unitary A*
  shows *A* ∈ *carrier-mat* (*dim-col A*) (*dim-col A*)
  ⟨*proof*⟩

lemma *unitary-carrier-square-row*:
  assumes *uA*: *unitary A*
  shows *A* ∈ *carrier-mat* (*dim-row A*) (*dim-row A*)
  ⟨*proof*⟩

lemma *unitary-dagger*:
  assumes *uA*: *unitary A*
  shows *unitary* ($A^\dagger$)
⟨*proof*⟩

lemma *unitary-mult*:
  assumes *UA*: *unitary A*
    and *UB*: *unitary B*
    and *dimA*: *A* ∈ *carrier-mat n n*
    and *dimB*: *B* ∈ *carrier-mat n n*
  shows *unitary* (*A* ∗ *B*)
  ⟨*proof*⟩

lemma *unitary-mult2*:
  assumes *UA*: *unitary A*
    and *UB*: *unitary B*
    and *dims*: *dim-row A* = *dim-row B*

shows *unitary* $(A * B)$
⟨*proof*⟩

9 Kronecker products

Kronecker products of matrices, $A \bigotimes B$: the tensor of identities, the adjoint $(R \otimes T)^* = R^* \otimes T^*$, the mixed-product property $(R_1 \otimes T_1)(R_2 \otimes T_2) = R_1 R_2 \otimes T_1 T_2$ (both from Day 9), and unitarity of tensors of unitaries.

9.1 Tensor of Identities

lemma *tensor-mat-id*:
Tensor.tensor-mat $(1_m\ n)\ (1_m\ m) = 1_m\ (n * m)$
⟨*proof*⟩

9.2 Tensor Adjoint

9.3 Mixed-Product Property

lemma *tensor-mat-mult*:
assumes *dimAC*: *dim-col A* = *dim-row C*
and *dimBD*: *dim-col B* = *dim-row D*
shows *Tensor.tensor-mat A B* ∗ *Tensor.tensor-mat C D* =
Tensor.tensor-mat $(A * C)\ (B * D)$
⟨*proof*⟩

9.4 Corollaries of the Mixed-Product Property

lemma *tensor-adjoint-mult*:
(*Tensor.tensor-mat* $(A^\dagger)\ (B^\dagger)) * (Tensor.tensor\text{-}mat\ A\ B) =$
Tensor.tensor-mat $((A^\dagger) * A)\ ((B^\dagger) * B)$
⟨*proof*⟩

lemma *tensor-mult-adjoint*:
(*Tensor.tensor-mat A B*) ∗ (*Tensor.tensor-mat* $(A^\dagger)\ (B^\dagger)) =$
Tensor.tensor-mat $(A * (A^\dagger))\ (B * (B^\dagger))$
⟨*proof*⟩

9.5 Tensor Unitarity

lemma *tensor-mat-unitary-carrier*:
assumes *dimA*: $A \in$ *carrier-mat n n*
and *dimB*: $B \in$ *carrier-mat m m*
and *uA*: *unitary A*
and *uB*: *unitary B*
shows *unitary* (*Tensor.tensor-mat A B*)
⟨*proof*⟩

lemma *tensor-mat-unitary*:
assumes *uA*: *unitary A*
and *uB*: *unitary B*
shows *unitary* (*Tensor.tensor-mat A B*)
⟨*proof*⟩

lemma *tensor-mat-unitary-one-left* [*simp*]:
assumes *uA*: *unitary A*
shows *unitary* (*Tensor.tensor-mat* $(1_m\ n)\ A$)

⟨*proof*⟩

lemma *tensor-mat-unitary-one-right* [*simp*]:
assumes *uA*: *unitary A*
shows *unitary* (*Tensor.tensor-mat A* (1_m *n*))
⟨*proof*⟩

lemma *one-smult-mat*: (*1*::*complex*) $\cdot_m$ *M* = *M*
⟨*proof*⟩

lemma *mult-index-lt*:
fixes *i j n m* :: *nat*
assumes *i* < *n* **and** *j* < *m*
shows *i* ∗ *m* + *j* < *n* ∗ *m*
⟨*proof*⟩

lemma *mult-index-div-mod*:
fixes *i l m* :: *nat*
assumes *l* < *m*
shows (*i* ∗ *m* + *l*) *div m* = *i* (*i* ∗ *m* + *l*) *mod m* = *l*
⟨*proof*⟩

9.6 Tensor products of matrices

lemma *tensor-index-c*:
fixes *A B* :: *complex Matrix.mat*
assumes *A* ∈ *carrier-mat r c B* ∈ *carrier-mat p q i* < *r* ∗ *p j* < *c* ∗ *q 0* < *c 0* < *q*
shows *Tensor.tensor-mat A B* $$ (*i*,*j*) = *A* $$ (*i div p*, *j div q*) ∗ *B* $$ (*i mod p*, *j mod q*)
⟨*proof*⟩

lemma *tensor-one-left*:
fixes *A* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat m m* **and** *m*: *0* < *m*
shows *Tensor.tensor-mat* (1_m *1*) *A* = *A*
⟨*proof*⟩

lemma *tensor-one-right*:
fixes *A* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat m m* **and** *m*: *0* < *m*
shows *Tensor.tensor-mat A* (1_m *1*) = *A*
⟨*proof*⟩

lemma *tensor-mat-add-right*:
fixes *A B C* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat p q* **and** *Cc*: *C* ∈ *carrier-mat p q*
and *c*: *0* < *c* **and** *q*: *0* < *q*
shows *Tensor.tensor-mat A* (*B* + *C*) = *Tensor.tensor-mat A B* + *Tensor.tensor-mat A C*
⟨*proof*⟩

lemma *tensor-mat-add-left*:
fixes *A B C* :: *complex Matrix.mat*
assumes *Ac*: *A* ∈ *carrier-mat r c* **and** *Bc*: *B* ∈ *carrier-mat r c* **and** *Cc*: *C* ∈ *carrier-mat p q*
and *c*: *0* < *c* **and** *q*: *0* < *q*
shows *Tensor.tensor-mat* (*A* + *B*) *C* = *Tensor.tensor-mat A C* + *Tensor.tensor-mat B C*
⟨*proof*⟩

lemma *tensor-mat-smult-right*:
fixes A B :: *complex Matrix.mat*
assumes *Ac*: $A \in$ *carrier-mat* r c **and** *Bc*: $B \in$ *carrier-mat* p q **and** *c*: $0 < c$ **and** *q*: $0 < q$
shows *Tensor.tensor-mat* A $(k \cdot_m B) = k \cdot_m$ *Tensor.tensor-mat* A B
⟨*proof*⟩

lemma *tensor-mat-smult-left*:
fixes A B :: *complex Matrix.mat*
assumes *Ac*: $A \in$ *carrier-mat* r c **and** *Bc*: $B \in$ *carrier-mat* p q **and** *c*: $0 < c$ **and** *q*: $0 < q$
shows *Tensor.tensor-mat* $(k \cdot_m A)$ $B = k \cdot_m$ *Tensor.tensor-mat* A B
⟨*proof*⟩

10 Typed-dimension matrix wrapper

The type ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* represents matrices of size $CARD('m) \times CARD('n)$ with entries in '*a*, by typedef of the JNF matrix type at the appropriate dimension.

typedef (**overloaded**) ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* =
$\{M$ :: '*a Matrix.mat*. *dim-row* $M = CARD('m) \wedge$ *dim-col* $M = CARD('n)\}$
morphisms *unwrap-lm wrap-lm*
⟨*proof*⟩

setup-lifting *type-definition-lmat*

lemma *unwrap-lm-dim-row* [*simp*]:
dim-row (*unwrap-lm* (M :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)) = $CARD('m)$
⟨*proof*⟩

lemma *unwrap-lm-dim-col* [*simp*]:
dim-col (*unwrap-lm* (M :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)) = $CARD('n)$
⟨*proof*⟩

lemma *unwrap-lm-carrier* [*simp*]:
unwrap-lm (M :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)
$\in$ *carrier-mat* $(CARD('m))$ $(CARD('n))$
⟨*proof*⟩

11 Typeclass instances

The unique zero matrix of the right shape.

instantiation *lmat* :: (*zero*, *finite*, *finite*) *zero*
begin
lift-definition *zero-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
is 0_m $(CARD('m))$ $(CARD('n))$ ⟨*proof*⟩
instance ⟨*proof*⟩
end

lemma *unwrap-lm-zero* [*simp*]:
unwrap-lm (0 :: ('*a* :: *zero*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)
$= 0_m$ $(CARD('m))$ $(CARD('n))$
⟨*proof*⟩

instantiation *lmat* :: (*plus*, *finite*, *finite*) *plus*
begin
lift-definition *plus-lmat* ::

('a, 'm :: finite, 'n :: finite) lmat ⇒ ('a, 'm :: finite, 'n :: finite) lmat ⇒ ('a, 'm :: finite, 'n :: finite) lmat
is (+) ⟨proof⟩
instance ⟨proof⟩
end

lemma *unwrap-lm-plus* [simp]:
unwrap-lm ((A :: ('a :: plus, 'm :: finite, 'n :: finite) lmat) + B)
= *unwrap-lm* A + *unwrap-lm* B
⟨proof⟩

instantiation lmat :: (uminus, finite, finite) uminus
begin
lift-definition *uminus-lmat* ::
('a, 'm :: finite, 'n :: finite) lmat ⇒ ('a, 'm :: finite, 'n :: finite) lmat
is *uminus* ⟨proof⟩
instance ⟨proof⟩
end

lemma *unwrap-lm-uminus* [simp]:
unwrap-lm (− (A :: ('a :: uminus, 'm :: finite, 'n :: finite) lmat))
= − *unwrap-lm* A
⟨proof⟩

instantiation lmat :: (minus, finite, finite) minus
begin
lift-definition *minus-lmat* ::
('a, 'm :: finite, 'n :: finite) lmat ⇒ ('a, 'm :: finite, 'n :: finite) lmat ⇒ ('a, 'm :: finite, 'n :: finite) lmat
is (−) ⟨proof⟩
instance ⟨proof⟩
end

lemma *unwrap-lm-minus* [simp]:
unwrap-lm ((A :: ('a :: minus, 'm :: finite, 'n :: finite) lmat) − B)
= *unwrap-lm* A − *unwrap-lm* B
⟨proof⟩

Lift the additive structure laws via ⟦⋀i j. ⟦i < dim-row ?B; j < dim-col ?B⟧ ⟹ ?A $$ (i, j) = ?B $$ (i, j); dim-row ?A = dim-row ?B; dim-col ?A = dim-col ?B⟧ ⟹ ?A = ?B on the underlying JNF matrices.

instance lmat :: (semigroup-add, finite, finite) semigroup-add
⟨proof⟩

instance lmat :: (ab-semigroup-add, finite, finite) ab-semigroup-add
⟨proof⟩

instance lmat :: (monoid-add, finite, finite) monoid-add
⟨proof⟩

instance lmat :: (comm-monoid-add, finite, finite) comm-monoid-add
⟨proof⟩

instance lmat :: (group-add, finite, finite) group-add
⟨proof⟩

instance lmat :: (ab-group-add, finite, finite) ab-group-add
⟨proof⟩

12 Real scalar multiplication and Hahn-Banach overloading

HOL's $(\cdot)$ is the abstract real-scalar multiplication used by the Hahn-Banach locales. We overload it on the type $(complex,\ 'm,\ 'n)\ lmat$ as multiplication by the embedded real, lifted from JNF's $(\cdot_m)$.

overloading
  prod-lmat ≡
    Vector-Space.prod ::
      real ⇒ (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
            ⇒ (*complex*, *'m*, *'n*) *lmat*
begin
definition *prod-lmat* ::
  real ⇒ (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
        ⇒ (*complex*, *'m*, *'n*) *lmat*
where
  prod-lmat r M = *wrap-lm* (*complex-of-real r* $\cdot_m$ *unwrap-lm M*)
end

lemma *unwrap-lm-prod* [*simp*]:
  unwrap-lm (*Vector-Space.prod r* (*M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*))
    = *complex-of-real r* $\cdot_m$ *unwrap-lm M*
  ⟨*proof*⟩

13 Real-vectorspace locale on the full type

Every $(complex,\ 'm,\ 'n)\ lmat$ with the universe set is a real vector space in HOL/Hahn_Banach's sense.

lemma *vectorspace-UNIV-lmat*:
  Vector-Space.vectorspace (*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*)
⟨*proof*⟩

14 The Hilbert-Schmidt norm on $(complex,\ 'm,\ 'n)\ lmat$

The Hilbert–Schmidt (Frobenius) norm $\|M\| = (\sum_{i,j} |M_{ij}|^2)^{1/2}$, the Euclidean norm of the entries.

definition *hs-norm-lmat* ::
    (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ *real*
where
  hs-norm-lmat M =
    L2-set (λ(*i*, *j*). *cmod* (*unwrap-lm M* $$ (*i*, *j*))) ({*0*..<*CARD*('*m*)} × {*0*..<*CARD*('*n*)})

lemma *hs-norm-lmat-nonneg*: *0* ≤ *hs-norm-lmat M*
  ⟨*proof*⟩

Scaling every entry in modulus by $r \geq 0$ scales the norm by r.

lemma *hs-norm-lmat-scale*:
  fixes *M N* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
  assumes ⋀*i j*. *i* < *CARD*('*m*) ⟹ *j* < *CARD*('*n*) ⟹
          cmod (*unwrap-lm N* $$ (*i*, *j*)) = *r* ∗ *cmod* (*unwrap-lm M* $$ (*i*, *j*))
    and *0* ≤ *r*
  shows *hs-norm-lmat N* = *r* ∗ *hs-norm-lmat M*
⟨*proof*⟩

lemma *hs-norm-lmat-smult*:

hs-norm-lmat (*Vector-Space.prod a M*) = |*a*| ∗ *hs-norm-lmat M*
⟨*proof*⟩

lemma *hs-norm-lmat-triangle*:
fixes *M N* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *hs-norm-lmat* (*M* + *N*) ≤ *hs-norm-lmat M* + *hs-norm-lmat N*
⟨*proof*⟩

15 Subspace, seminorm, normed-vectorspace instances

HS norm is a seminorm on the whole real vector space of (*complex*, *'m*, *'n*) *lmat*.

lemma *seminorm-UNIV-hs-norm-lmat*:
seminorm (*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*) *hs-norm-lmat*
⟨*proof*⟩

HS norm is a (real) norm: it vanishes only on the zero matrix.

lemma *hs-norm-lmat-zero-iff*:
fixes *M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows (*hs-norm-lmat M* = *0*) ⟷ (*M* = *0*)
⟨*proof*⟩

lemma *norm-UNIV-hs-norm-lmat*:
Normed-Space.norm
(*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*) *hs-norm-lmat*
⟨*proof*⟩

lemma *normed-vectorspace-UNIV-lmat*:
normed-vectorspace
(*UNIV* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*) *hs-norm-lmat*
⟨*proof*⟩

A complex-linear subspace V of (*complex*, *'m*, *'n*) *lmat* (i.e. nonempty, closed under addition and complex-scalar multiplication) is in particular a real-linear subspace, hence a *subspace* of *UNIV* in HOL/Hahn_Banach's sense.

definition *cmat-subspace-lmat* ::
(*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set* ⇒ *bool*
where
cmat-subspace-lmat V ≡
V ≠ {} ∧
(∀ *x* ∈ *V*. ∀ *y* ∈ *V*. *x* + *y* ∈ *V*) ∧
(∀ *x* ∈ *V*. ∀ *c* :: *complex*.
wrap-lm (*c* ·$_m$ *unwrap-lm x*) ∈ *V*)

lemma *subspace-of-cmat-subspace-lmat*:
fixes *V* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
assumes *cmat-subspace-lmat V*
shows *Subspace.subspace V* (*UNIV* :: (*complex*, *'m*, *'n*) *lmat set*)
⟨*proof*⟩

16 Complex scalar multiplication on lmat

Complex scalar multiplication on (*complex*, *'m*, *'n*) *lmat*, extending the *Vector-Space.prod* overload (which is only by reals).

definition *cmult-lmat* ::

complex ⇒ (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
⇒ (*complex*, *'m*, *'n*) *lmat*
where
cmult-lmat c M = *wrap-lm* (*c* ·$_m$ *unwrap-lm M*)

lemma *unwrap-lm-cmult* [*simp*]:
unwrap-lm (*cmult-lmat c* (*M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*))
= *c* ·$_m$ *unwrap-lm M*
⟨*proof*⟩

lemma *cmult-lmat-real*:
cmult-lmat (*complex-of-real a*) *M* = *Vector-Space.prod a M*
⟨*proof*⟩

lemma *cmult-lmat-add*:
fixes *x y* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *cmult-lmat c* (*x* + *y*) = *cmult-lmat c x* + *cmult-lmat c y*
⟨*proof*⟩

lemma *cmult-lmat-scalar-add*:
fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *cmult-lmat* (*c* + *d*) *x* = *cmult-lmat c x* + *cmult-lmat d x*
⟨*proof*⟩

lemma *cmult-lmat-scalar-mult*:
fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *cmult-lmat* (*c* ∗ *d*) *x* = *cmult-lmat c* (*cmult-lmat d x*)
⟨*proof*⟩

lemma *cmult-lmat-one*:
fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *cmult-lmat 1 x* = *x*
⟨*proof*⟩

lemma *cmult-lmat-uminus-one*:
fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *cmult-lmat* (− *1*) *x* = − *x*
⟨*proof*⟩

lemma *cmult-lmat-i-sq*:
fixes *x* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *cmult-lmat* i (*cmult-lmat* i *x*) = − *x*
⟨*proof*⟩

lemma *hs-norm-lmat-cmult*:
fixes *M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *hs-norm-lmat* (*cmult-lmat c M*) = *cmod c* ∗ *hs-norm-lmat M*
⟨*proof*⟩

lemma *hs-norm-lmat-cmult-i* [*simp*]:
fixes *M* :: (*complex*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
shows *hs-norm-lmat* (*cmult-lmat* i *M*) = *hs-norm-lmat M*
⟨*proof*⟩

The HOL/Hahn_Banach development imports locales also called *norm*, *seminorm* and *continuous*. Hide their short names so that *norm* is the class method from *HOL.Real-Vector-Spaces* and *continuous* the topological predicate.

hide-const (**open**) *Normed-Space.norm*
hide-const (**open**) *Normed-Space.seminorm*
hide-const (**open**) *Function-Norm.continuous*

17 Overview

Nothing in this theory is stated in the notes; it is the bridge that lets the HOL-Analysis theorems on convex sets (separating hyperplanes, compactness) be applied to matrices. (*'a*, *'m*, *'n*) *lmat* is given the typeclass tower *real-vector*, *metric-space*, *real-normed-vector*, *real-inner*, *euclidean-space*, *heine-borel*; for *'a* = *complex* the norm is the Hilbert–Schmidt norm, the inner product its real part, and the dimension is $2\,\mathrm{CARD}('m)\,\mathrm{CARD}('n)$.

18 Real scalar multiplication and *real-vector* instance

For *'a* :: *real-vector*, lift element-wise scalar multiplication via *map-mat* (*scaleR a*).

instantiation *lmat* :: (*real-vector*, *finite*, *finite*) *scaleR*
begin

lift-definition *scaleR-lmat* ::
real ⇒ (*'a*, *'m* :: *finite*, *'n* :: *finite*) *lmat* ⇒ (*'a*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
is λ*r M*. *map-mat* (λ*x*. *r* ∗$_R$ *x*) *M* ⟨*proof*⟩

instance ⟨*proof*⟩

end

lemma *unwrap-lm-scaleR* [*simp*]:
unwrap-lm ((*r* :: *real*) ∗$_R$ (*M* :: (*'a* :: *real-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat*))
= *map-mat* (λ*x*. *r* ∗$_R$ *x*) (*unwrap-lm M*)
⟨*proof*⟩

lemma *unwrap-lm-scaleR-index* [*simp*]:
fixes *M* :: (*'a* :: *real-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
assumes *i* < *CARD*(*'m*) *j* < *CARD*(*'n*)
shows *unwrap-lm* (*r* ∗$_R$ *M*) $$ (*i*, *j*) = *r* ∗$_R$ (*unwrap-lm M* $$ (*i*, *j*))
⟨*proof*⟩

instance *lmat* :: (*real-vector*, *finite*, *finite*) *real-vector*
⟨*proof*⟩

19 Entries of sums and differences

lemma *unwrap-lm-plus-index* [*simp*]:
fixes *M N* :: (*'a* :: *plus*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
assumes *i* < *CARD*(*'m*) *j* < *CARD*(*'n*)
shows *unwrap-lm* (*M* + *N*) $$ (*i*, *j*) = *unwrap-lm M* $$ (*i*, *j*) + *unwrap-lm N* $$ (*i*, *j*)
⟨*proof*⟩

lemma *unwrap-lm-minus-index* [*simp*]:
fixes *M N* :: (*'a* :: *ab-group-add*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
assumes *i* < *CARD*(*'m*) *j* < *CARD*(*'n*)
shows *unwrap-lm* (*M* − *N*) $$ (*i*, *j*) = *unwrap-lm M* $$ (*i*, *j*) − *unwrap-lm N* $$ (*i*, *j*)
⟨*proof*⟩

lemma *unwrap-lm-zero-index* [*simp*]:
fixes $i\ j :: nat$
assumes $i < CARD('m :: finite)\ j < CARD('n :: finite)$
shows $unwrap\text{-}lm\ (0 :: ('a :: zero, 'm, 'n)\ lmat)\ \$\$\ (i, j) = 0$
⟨*proof*⟩

lemma *lmat-eq-iff-entries*:
fixes $M\ N :: ('a, 'm :: finite, 'n :: finite)\ lmat$
shows $M = N \longleftrightarrow$
$(\forall\, i\ j.\ i < CARD('m) \longrightarrow j < CARD('n)$
$\longrightarrow unwrap\text{-}lm\ M\ \$\$\ (i, j) = unwrap\text{-}lm\ N\ \$\$\ (i, j))$
⟨*proof*⟩

20 Distance, uniformity, open: *metric-space* instance

Distance is the L2 sum of entry-wise distances, indexed by $\{0..<CARD('m)\} \times \{0..<CARD('n)\}$. Uniformity and *open* are defined via the standard metric-induced filter so that the *open-uniformity* axiom is discharged by reflexivity.

instantiation *lmat* :: (*metric-space*, *finite*, *finite*) *dist*
begin

definition *dist-lmat* :: $('a, 'm :: finite, 'n :: finite)\ lmat$
$\Rightarrow ('a, 'm :: finite, 'n :: finite)\ lmat \Rightarrow real$ **where**
$dist\text{-}lmat\ M\ N = L2\text{-}set\ (\lambda(i, j).\ dist\ (unwrap\text{-}lm\ M\ \$\$\ (i, j))\ (unwrap\text{-}lm\ N\ \$\$\ (i, j)))$
$(\{0..<CARD('m)\} \times \{0..<CARD('n)\})$

instance ⟨*proof*⟩

end

instantiation *lmat* :: (*metric-space*, *finite*, *finite*) *uniformity-dist*
begin

definition *uniformity-lmat* ::
$(('a, 'm :: finite, 'n :: finite)\ lmat \times ('a, 'm, 'n)\ lmat)\ filter$ **where**
$(uniformity\text{-}lmat :: (('a, 'm, 'n)\ lmat \times ('a, 'm, 'n)\ lmat)\ filter)$
$= (INF\ e \in \{0<..\}.\ principal\ \{(x, y).\ dist\ x\ y < e\})$

instance ⟨*proof*⟩

end

instantiation *lmat* :: (*metric-space*, *finite*, *finite*) *metric-space*
begin

definition *open-lmat* :: $('a, 'm :: finite, 'n :: finite)\ lmat\ set \Rightarrow bool$ **where**
$open\text{-}lmat\ U = (\forall\, x \in U.\ \forall_F\ (x', y)\ in\ uniformity.\ x' = x \longrightarrow y \in U)$

instance ⟨*proof*⟩

end

21 Norm and *real-normed-vector* instance

instantiation *lmat* :: (*real-normed-vector*, *finite*, *finite*) *real-normed-vector*
begin

definition *norm-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* ⇒ *real* **where**
norm-lmat M = *L2-set* (λ(*i*, *j*). *norm* (*unwrap-lm M* $$ (*i*, *j*)))
({*0*..<*CARD*('*m*)} × {*0*..<*CARD*('*n*)})

definition *sgn-lmat* ::
('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* ⇒ ('*a*, '*m*, '*n*) *lmat* **where**
sgn-lmat (*M* :: ('*a*, '*m*, '*n*) *lmat*) = *scaleR* (*inverse* (*norm M*)) *M*

instance ⟨*proof*⟩

end

22 Real inner product

For '*a* :: *real-inner*, the lmat inner product is the entry-wise sum of inner products. When '*a* = *complex* with the standard real-inner instance *inner z w* = *Re* (*cnj z* ∗ *w*), this is the real part of the Hilbert-Schmidt inner product.

instantiation *lmat* :: (*real-inner*, *finite*, *finite*) *real-inner*
begin

definition *inner-lmat* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
⇒ ('*a*, '*m*, '*n*) *lmat* ⇒ *real* **where**
inner-lmat M N = (∑ (*i*, *j*) ∈ {*0*..<*CARD*('*m*)} × {*0*..<*CARD*('*n*)}.
inner (*unwrap-lm M* $$ (*i*, *j*)) (*unwrap-lm N* $$ (*i*, *j*)))

instance ⟨*proof*⟩

end

23 Basis: matrix units carrying a basis vector

definition *lmat-axis* :: *nat* ⇒ *nat* ⇒ '*a* :: *zero*
⇒ ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat* **where**
lmat-axis i j u = *wrap-lm* (*Matrix.mat* (*CARD*('*m*)) (*CARD*('*n*))
(λ(*r*, *s*). *if r* = *i* ∧ *s* = *j then u else 0*))

lemma *unwrap-lmat-axis* [*simp*]:
unwrap-lm (*lmat-axis i j u* :: ('*a* :: *zero*, '*m* :: *finite*, '*n* :: *finite*) *lmat*)
= *Matrix.mat* (*CARD*('*m*)) (*CARD*('*n*))
(λ(*r*, *s*). *if r* = *i* ∧ *s* = *j then u else 0*)
⟨*proof*⟩

lemma *lmat-axis-index* [*simp*]:
fixes *u* :: '*a* :: *zero*
assumes *r* < *CARD*('*m* :: *finite*) *s* < *CARD*('*n* :: *finite*)
shows *unwrap-lm* (*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*) $$ (*r*, *s*)
= (*if r* = *i* ∧ *s* = *j then u else 0*)
⟨*proof*⟩

Inner product against a single matrix-unit basis vector picks out one matrix entry.

lemma *inner-lmat-axis*:
fixes *u* :: '*a* :: *real-inner*
and *M* :: ('*a*, '*m* :: *finite*, '*n* :: *finite*) *lmat*
assumes *ij*: *i* < *CARD*('*m*) *j* < *CARD*('*n*)
shows *inner M* (*lmat-axis i j u* :: ('*a*, '*m*, '*n*) *lmat*)

$= \mathit{inner}\ (\mathit{unwrap\text{-}lm}\ M\ \$\$\ (i, j))\ u$
⟨*proof*⟩

Two matrix-unit basis vectors are equal iff their entries and positions match (or both vectors are zero).

lemma *lmat-axis-eq-iff*:
fixes $u\ v :: 'a :: \mathit{zero}$
assumes $i < \mathit{CARD}('m :: \mathit{finite})\ j < \mathit{CARD}('n :: \mathit{finite})$
and $i' < \mathit{CARD}('m)\ j' < \mathit{CARD}('n)$
shows $(\mathit{lmat\text{-}axis}\ i\ j\ u :: ('a, 'm, 'n)\ \mathit{lmat}) = \mathit{lmat\text{-}axis}\ i'\ j'\ v$
$\longleftrightarrow (u = v \wedge i = i' \wedge j = j') \vee (u = 0 \wedge v = 0)$
⟨*proof*⟩

Inner product between two matrix-unit basis vectors: 1 if same position and basis component, 0 otherwise.

lemma *inner-lmat-axis-axis*:
fixes $u\ v :: 'a :: \mathit{real\text{-}inner}$
assumes *ij*: $i < \mathit{CARD}('m :: \mathit{finite})\ j < \mathit{CARD}('n :: \mathit{finite})$
and *ij'*: $i' < \mathit{CARD}('m)\ j' < \mathit{CARD}('n)$
shows $\mathit{inner}\ (\mathit{lmat\text{-}axis}\ i\ j\ u :: ('a, 'm, 'n)\ \mathit{lmat})\ (\mathit{lmat\text{-}axis}\ i'\ j'\ v)$
$= (\mathit{if}\ i = i' \wedge j = j'\ \mathit{then}\ \mathit{inner}\ u\ v\ \mathit{else}\ 0)$
⟨*proof*⟩

24 Basis set

definition *Basis-lmat-set* :: $('a :: \mathit{euclidean\text{-}space}, 'm :: \mathit{finite}, 'n :: \mathit{finite})\ \mathit{lmat}\ \mathit{set}$
where $\mathit{Basis\text{-}lmat\text{-}set} = (\bigcup i \in \{0..<\mathit{CARD}('m)\}.$
$\bigcup j \in \{0..<\mathit{CARD}('n)\}.$
$\bigcup u \in \mathit{Basis}.\ \{\mathit{lmat\text{-}axis}\ i\ j\ u :: ('a, 'm, 'n)\ \mathit{lmat}\})$

lemma *Basis-lmat-set-finite*:
$\mathit{finite}\ (\mathit{Basis\text{-}lmat\text{-}set} :: ('a :: \mathit{euclidean\text{-}space}, 'm :: \mathit{finite}, 'n :: \mathit{finite})\ \mathit{lmat}\ \mathit{set})$
⟨*proof*⟩

lemma *Basis-lmat-set-nonempty*:
$\mathit{Basis\text{-}lmat\text{-}set} \neq (\{\} :: ('a :: \mathit{euclidean\text{-}space}, 'm :: \mathit{finite}, 'n :: \mathit{finite})\ \mathit{lmat}\ \mathit{set})$
⟨*proof*⟩

lemma *Basis-lmat-orthonormal*:
fixes $u\ v :: ('a :: \mathit{euclidean\text{-}space}, 'm :: \mathit{finite}, 'n :: \mathit{finite})\ \mathit{lmat}$
assumes *uB*: $u \in \mathit{Basis\text{-}lmat\text{-}set}$ **and** *vB*: $v \in \mathit{Basis\text{-}lmat\text{-}set}$
shows $\mathit{inner}\ u\ v = (\mathit{if}\ u = v\ \mathit{then}\ 1\ \mathit{else}\ 0)$
⟨*proof*⟩

lemma *axis-mem-Basis-lmat-set*:
fixes $u :: 'a :: \mathit{euclidean\text{-}space}$
assumes *i-lt*: $i < \mathit{CARD}('m :: \mathit{finite})$
and *j-lt*: $j < \mathit{CARD}('n :: \mathit{finite})$
and *uB*: $u \in \mathit{Basis}$
shows $(\mathit{lmat\text{-}axis}\ i\ j\ u :: ('a, 'm, 'n)\ \mathit{lmat}) \in \mathit{Basis\text{-}lmat\text{-}set}$
⟨*proof*⟩

lemma *Basis-lmat-perp-imp-zero*:
fixes $x :: ('a :: \mathit{euclidean\text{-}space}, 'm :: \mathit{finite}, 'n :: \mathit{finite})\ \mathit{lmat}$
assumes $*$: $\forall u \in \mathit{Basis\text{-}lmat\text{-}set}.\ \mathit{inner}\ x\ u = 0$
shows $x = 0$
⟨*proof*⟩

lemma *zero-inner-Basis-lmat*:
fixes $u :: ('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat$
shows $inner\ (0 :: ('a, 'm, 'n)\ lmat)\ u = 0$
⟨*proof*⟩

lemma *Basis-lmat-perp-zero*:
fixes $x :: ('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat$
shows $(\forall u \in Basis\text{-}lmat\text{-}set.\ inner\ x\ u = 0) \longleftrightarrow x = 0$
⟨*proof*⟩

25 Euclidean space instance

instantiation $lmat :: (euclidean\text{-}space, finite, finite)\ euclidean\text{-}space$
begin

definition $Basis\text{-}lmat :: ('a, 'm :: finite, 'n :: finite)\ lmat\ set$ **where**
$Basis\text{-}lmat = Basis\text{-}lmat\text{-}set$

instance ⟨*proof*⟩

end

26 Heine-Borel

instance $lmat :: (euclidean\text{-}space, finite, finite)\ heine\text{-}borel$ ⟨*proof*⟩

27 Dimension

Two distinct (i, j, u) with $u \in Basis\text{-}'a$ give distinct matrix-axis vectors.

lemma *lmat-axis-inj-on-Basis-indices*:
fixes $u\ v :: 'a :: euclidean\text{-}space$
assumes *ij*: $i < CARD('m :: finite)\ j < CARD('n :: finite)$
and *ij'*: $i' < CARD('m)\ j' < CARD('n)$
and *uB*: $u \in Basis$ **and** *vB*: $v \in Basis$
shows $(lmat\text{-}axis\ i\ j\ u :: ('a, 'm, 'n)\ lmat) = lmat\text{-}axis\ i'\ j'\ v$
$\longleftrightarrow i = i' \wedge j = j' \wedge u = v$
⟨*proof*⟩

lemma *DIM-lmat*:
$DIM(('a :: euclidean\text{-}space, 'm :: finite, 'n :: finite)\ lmat)$
$= CARD('m) * CARD('n) * DIM('a)$
⟨*proof*⟩

28 Component-wise bounds via the L2 norm

lemma *norm-lmat-index-le*:
fixes $M :: ('a :: real\text{-}normed\text{-}vector, 'm :: finite, 'n :: finite)\ lmat$
assumes $i < CARD('m)\ j < CARD('n)$
shows $norm\ (unwrap\text{-}lm\ M\ \$\$\ (i, j)) \leq norm\ M$
⟨*proof*⟩

lemma *dist-lmat-index-le*:
fixes $M\ N :: ('a :: real\text{-}normed\text{-}vector, 'm :: finite, 'n :: finite)\ lmat$
assumes $i < CARD('m)\ j < CARD('n)$

shows *dist* (*unwrap-lm M $$ (i, j)*) (*unwrap-lm N $$ (i, j)*) ≤ *dist M N*
⟨*proof*⟩

29 Convergence: lmat sequence convergence implies entry convergence

lemma *tendsto-unwrap-lm-index*:
fixes *f* :: *'b* ⇒ (*'a* :: *real-normed-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
and *L* :: (*'a*, *'m*, *'n*) *lmat*
assumes *lim*: (*f* ⟶ *L*) *F*
and *ij*: *i* < *CARD*(*'m*) *j* < *CARD*(*'n*)
shows ((λ*x*. *unwrap-lm* (*f x*) $$ (*i*, *j*)) ⟶ *unwrap-lm L $$ (i, j)*) *F*
⟨*proof*⟩

lemma *LIMSEQ-unwrap-lm-index*:
fixes *f* :: *nat* ⇒ (*'a* :: *real-normed-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
and *L* :: (*'a*, *'m*, *'n*) *lmat*
assumes *f* ⟶ *L i* < *CARD*(*'m*) *j* < *CARD*(*'n*)
shows (λ*n*. *unwrap-lm* (*f n*) $$ (*i*, *j*)) ⟶ *unwrap-lm L $$ (i, j)*
⟨*proof*⟩

30 Entries as continuous functions; wrapping

The entry-projection $M \mapsto$ *unwrap-lm M $$ (i, j)* is continuous on lmat for valid index (i, j).

lemma *isCont-unwrap-lm-index*:
fixes *x* :: (*'a* :: *real-normed-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat*
assumes *ij*: *i* < *CARD*(*'m*) *j* < *CARD*(*'n*)
shows *isCont* (λ*M*. *unwrap-lm M $$ (i, j)*) *x*
⟨*proof*⟩

lemma *continuous-on-unwrap-lm-index*:
fixes *S* :: (*'a* :: *real-normed-vector*, *'m* :: *finite*, *'n* :: *finite*) *lmat set*
assumes *ij*: *i* < *CARD*(*'m*) *j* < *CARD*(*'n*)
shows *continuous-on S* (λ*M*. *unwrap-lm M $$ (i, j)*)
⟨*proof*⟩

lemma *unwrap-wrap-lm*:
fixes *M* :: *complex Matrix.mat*
assumes *M* ∈ *carrier-mat CARD*('*k*::*finite*) *CARD*(*'k*)
shows *unwrap-lm* (*wrap-lm M* :: (*complex*, *'k*, *'k*) *lmat*) = *M*
⟨*proof*⟩

lemma *unwrap-lm-scaleR-mat*:
fixes *R* :: (*complex*, *'k* :: *finite*, *'k*) *lmat*
shows *unwrap-lm* (*u* ∗$_R$ *R*) = *complex-of-real u* ·$_m$ *unwrap-lm R*
⟨*proof*⟩